

CODEIUM SECURITY

Intelligent Document Processing (IDP) Engine

Dossier de projet complet — de l'idée initiale à l'état actuel

Développeur : Samar

Stack : C# / ASP.NET Core / Tesseract OCR / ML.NET (à venir)

Statut : Sprint 2 en cours — Architecture IDP générique

Ce document résume l'ensemble du projet : l'idée de départ, pourquoi il a été lancé, comment il a évolué techniquement, ce qui fonctionne aujourd'hui, ce qui reste à faire, et la feuille de route vers le Machine Learning.

1. L'idée et l'objectif du projet

L'entreprise possédait déjà une application utilisant un algorithme d'IA/LLM pour extraire des informations depuis des documents bancaires, mais les résultats obtenus n'étaient pas satisfaisants.

Décision prise : ne pas créer un moteur OCR à partir de zéro, mais intégrer un moteur OCR local existant, puis construire une nouvelle chaîne de traitement plus fiable pour remplacer l'algorithme existant.

Pourquoi un OCR 100% local ?

- Aucune dépendance à un service externe (pas d'API tierce comme Google Vision, Azure OCR ou OCR.Space)
- Fonctionne hors ligne, sans limite d'appels, sans coût mensuel récurrent
- Respect de la confidentialité des données bancaires des clients
- Solution pérenne : doit continuer à fonctionner dans 10 ans, indépendamment de tout fournisseur externe

Choix technique retenu : Tesseract OCR

Tesseract a été choisi car il est gratuit, open source, 100% local, stable, et largement utilisé dans le monde. Il supporte plusieurs langues simultanément (français, anglais, arabe), ce qui est essentiel pour des documents bancaires tunisiens souvent bilingues.

Ce que le projet N'EST PAS

- Ce n'est pas un simple "OCR bancaire" limité à une seule banque
- Ce n'est pas un projet spécifique à BNA ou à une autre banque en particulier
- Ce n'est pas un système basé sur un LLM externe pour l'extraction finale

Vision finale : construire un véritable moteur IDP (Intelligent Document Processing), générique et commercialisable, capable de lire des relevés bancaires, factures, reçus, bons de livraison, documents administratifs et comptables. Le relevé bancaire n'est que le premier cas d'étude.

2. Stack technique

Composant	Choix	Rôle
Langage	C#	Toute la logique métier et le Machine Learning
Framework API	ASP.NET Core / .NET 8 (LTS)	Exposer un service web (Web API)
OCR	Tesseract 5.2.0	Lecture locale du texte dans les images
Documentation API	Swagger / Swashbuckle	Interface de test interactive
Conversion PDF	Docnet.Core (PDFium)	Convertir chaque page PDF en image avant OCR
Versionnage	Git / GitHub	Sauvegarde et historique du code

ML (à venir)	ML.NET	Correction et classification intelligente
--------------	--------	---

3. Architecture actuelle du pipeline

Le projet a évolué d'un simple appel OCR vers une véritable chaîne de traitement modulaire, inspirée du pattern **Factory** : un classifieur détecte le type de document, puis une usine (Factory) choisit automatiquement le bon parser spécialisé.



Organisation des fichiers du projet

```

Codeium Security
■■■■ Controllers/OcrController.cs
■■■■ Factories/DocumentParserFactory.cs
■■■■ Interfaces/IDocumentParser.cs
■■■■ Models/ (BankDocument, InvoiceDocument, ReceiptDocument,
■ AcademicDocument, Transaction, DocumentType)
■■■■ OCR/ (IOcrService, OcrResult, OcrWord, TesseractOcrService)
■■■■ Services/
■ ■■■■ DocumentClassification/ (IDocumentClassifier,
■ ■ KeywordDocumentClassifier)
■ ■■■■ DocumentParsers/ (BankDocumentParser, InvoiceDocumentParser,
■ ■ ReceiptDocumentParser, AcademicDocumentParser)
■ ■■■■ DocumentAnalysisEngine.cs
■ ■■■■ PdfToImageConverter.cs
■ ■■■■ ImageFormatConverter.cs

```

```
■■■ Utilities/TextCleaner.cs
■■■ tessdata/ (eng, fra, ara)
■■■ TrainingData/
■■■ Program.cs
```

4. Le chemin parcouru (chronologie réelle)

Étape 1 — Mise en place de l'environnement

Installation de Visual Studio, .NET, création du projet ASP.NET Core Web API, activation de Swagger UI pour pouvoir tester l'API visuellement sans écrire de client séparé.

Étape 2 — Premier pipeline OCR fonctionnel

Intégration de Tesseract, premier test réussi sur une image bancaire réelle (BNA). Découverte que le texte brut de l'OCR est imparfait (~79-91% de confiance) et nécessite un traitement supplémentaire.

Étape 3 — BankDocumentParser (première version)

Extraction par expressions régulières (regex) des champs clés : numéro de compte, devise, solde, nom du client, nom de la banque. Nombreux bugs corrigés un par un : mauvais nombre capturé pour le solde, signe négatif perdu, espaces mal gérés, champ vide à cause de caractères OCR imprévus.

Étape 4 — Extraction structurée (position des mots)

Ajout de la capture des coordonnées (X, Y) de chaque mot OCR, permettant de regrouper les mots en lignes logiques, puis de tenter une reconstruction de tableau (colonnes Date / Libellé / Débit / Crédit / Solde).

Étape 5 — Extraction des transactions

Passage d'une approche par "colonnes rigides" (fragile face aux espacements irréguliers de l'OCR) à une approche par regex sur le texte complet de chaque ligne — bien plus robuste. Premières transactions correctement extraites sur un document BNA.

Étape 6 — Support multi-fichiers et PDF multi-pages

Ajout de la détection automatique image/PDF, conversion PDF → images page par page via Docnet.Core, fusion des résultats OCR de plusieurs pages en un seul document cohérent (avec gestion du décalage vertical entre pages).

Étape 7 — Architecture générique (Factory Pattern)

Refonte pour ne plus dépendre d'un seul parser bancaire : création d'une interface commune IDocumentParser, d'un classifieur de document par mots-clés, et d'une Factory qui choisit automatiquement le bon parser (Bank, Invoice, Receipt, Academic).

Étape 8 — Généralisation multi-banques

Test avec un second établissement (BIAT). Découverte que chaque banque utilise un format de date et de montant différent (séparateurs de milliers/décimales inversés, dates collées sans séparateur). Le parser a été rendu tolérant à ces variations. Correction également d'un bug lié aux caractères Unicode invisibles générés par le texte bilingue arabe/français.

5. État actuel du projet

Brique	Avancement	Commentaire
Infrastructure (VS, .NET, Git)	100%	Terminé et stable
OCR local (Tesseract, 3 langues)	100%	Fonctionnel sur image et PDF
API + Swagger	100%	Multi-fichiers supporté
Lecture PDF multi-pages	90%	Fonctionnel, à tester en masse
Formats d'image non standards (webp...)	90%	Conversion automatique ajoutée
Classification de document	70%	Basée sur mots-clés, à raffiner
Parser bancaire (BankDocumentParser)	85%	Généralisé à 2 banques testées
Parsers Invoice / Receipt / Academic	30%	Versions basiques, pas encore testées en réel
Mesure de qualité sur un grand volume	0%	Prochaine étape prioritaire
Machine Learning (ML.NET)	0%	En attente de données d'entraînement

Ce qui a été appris (limites techniques honnêtes)

- Aucun moteur OCR — Tesseract ou une solution commerciale — n'atteint 100% de fiabilité sur des scans réels. Un taux de confiance de 79-95% est normal et attendu.
- Chaque banque utilise un format différent (dates, séparateurs décimaux). Chaque nouveau format nécessite un ajustement ciblé, ce qui est normal dans ce type de projet.
- Un système professionnel de ce type inclut toujours un score de confiance et, idéalement, une validation humaine pour les cas incertains — pas une confiance aveugle à 100% dans l'automatisation.

6. Prochaines étapes détaillées

Étape A — Traitement en masse des documents disponibles

Le Drive contient environ 56 à 58 PDF de banques différentes. Objectif : les faire passer un par un (ou par lots de 5-10) dans l'API actuelle, et sauvegarder chaque résultat JSON dans TrainingData/RawResults/.

Étape B — Créer un fichier de vérité terrain (ground truth)

Pour chaque document traité, noter à la main les vraies valeurs correctes (numéro de compte, solde, nombre de transactions...) dans un fichier ground_truth.csv. C'est la référence qui permettra de mesurer objectivement la qualité du système.

Étape C — Mesurer le taux de réussite par champ et par banque

Comparer les résultats du système avec la vérité terrain. Cela révèle précisément où concentrer les efforts : telle banque a un taux d'erreur élevé sur tel champ → prioriser son ajustement.

Étape D — Ajouter un score de confiance global par document

Ajouter un champ (ex. needsReview: true/false) dans le JSON de sortie, pour signaler automatiquement les documents à faire vérifier par un humain plutôt que de leur faire une confiance aveugle.

Étape E — Enrichir les parsers Invoice, Receipt et Academic

Actuellement basiques (V1). Une fois de vrais exemples de factures/reçus disponibles, les affiner de la même façon que le parser bancaire (regex robustes, gestion des formats variés).

Étape F — Démarrer le Machine Learning (ML.NET)

Une fois un volume suffisant de données annotées disponible (issu des étapes A à C), commencer par un modèle simple de classification de ligne ("cette ligne est-elle une transaction ?"), puis un modèle de classification de champ ("ce texte est-il une date, un montant, une description ?"). Le ML remplacera progressivement les règles regex les plus fragiles, sans jamais nécessiter une image en entrée (il travaillera uniquement sur des données déjà structurées par l'OCR et les parsers).

Principe directeur pour la suite : avancer par mesure et non par correction infinie au cas par cas. Chaque nouvelle banque testée peut révéler un nouveau format — c'est normal. L'objectif réaliste n'est pas "zéro erreur, quel que soit le fichier", mais un taux d'erreur mesuré, faible, avec un mécanisme de détection des cas incertains.